

Maps



Jon Friskics

Senior Software Development Author, Pluralsight

@jonfriskics@mastodon.social | www.pluralsight.com



JavaScript Maps: Create a Map

```
let weatherData = new Map();  
weatherData.set('Orlando', {"temp": 85, "condition": "Cloudy"});  
weatherData.set('New York', {"temp": 75, "condition": "Sunny"});
```

weatherData

key	value
Orlando	{"temp": 85, "condition": "Cloudy"}
New York	{"temp": 75, "condition": "Sunny"}



JavaScript Maps: Access Data by Key

```
let weatherData = new Map();  
weatherData.set('Orlando', {"temp": 85, "condition": "Cloudy"});  
weatherData.set('New York', {"temp": 75, "condition": "Sunny"});  
let data = weatherData.get('Orlando');  
// data == {"temp": 85, "condition": "Cloudy"}
```

weatherData

key	value
Orlando	{"temp": 85, "condition": "Cloudy"}
New York	{"temp": 75, "condition": "Sunny"}



JavaScript Maps: Access Data by Key

```
let weatherData = new Map();
weatherData.set('Orlando', {"temp": 85, "condition": "Cloudy"});
weatherData.set('New York', {"temp": 75, "condition": "Sunny"});
let data = weatherData.get('Atlanta');
// data == undefined
```

weatherData

key	value
Orlando	{"temp": 85, "condition": "Cloudy"}
New York	{"temp": 75, "condition": "Sunny"}



JavaScript Maps: Remove Data by Key

```
let weatherData = new Map();  
weatherData.set('Orlando', {"temp": 85, "condition": "Cloudy"});  
weatherData.set('New York', {"temp": 75, "condition": "Sunny"});  
  
weatherData.remove('New York');
```

weatherData

key	value
Orlando	{"temp": 85, "condition": "Cloudy"}
New York	{"temp": 75, "condition": "Sunny"}



JavaScript Maps: Check Size and if Key Exists

```
let weatherData = new Map();  
weatherData.set('Orlando', {"temp": 85, "condition": "Cloudy"});  
weatherData.set('New York', {"temp": 75, "condition": "Sunny"});  
weatherData.size; // 2  
weatherData.has('Atlanta'); // false  
weatherData.has('Orlando'); // true
```

weatherData

key	value
Orlando	{"temp": 85, "condition": "Cloudy"}
New York	{"temp": 75, "condition": "Sunny"}



JavaScript Maps: Iteration with for..of

```
let weatherData = new Map();  
weatherData.set('Orlando', {"temp": 85, "condition": "Cloudy"});  
weatherData.set('New York', {"temp": 75, "condition": "Sunny"});
```

```
for(let [key, value] of weatherData) {  
  console.log(key, value);  
};
```

.keys()
or
.values()

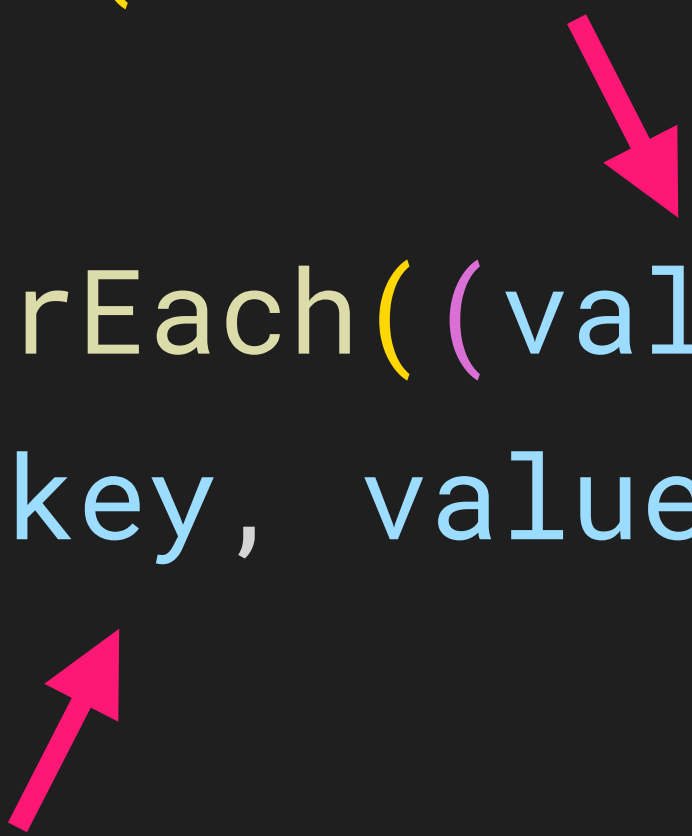
```
// logs these on a new line:  
// Orlando {"temp": 85, "condition": "Cloudy"}  
// New York {"temp": 75, "condition": "Sunny"}
```


JavaScript Maps: Iteration with foreach

```
let weatherData = new Map();
weatherData.set('Orlando', {"temp": 85, "condition": "Cloudy"});
weatherData.set('New York', {"temp": 75, "condition": "Sunny"});

weatherData.forEach((value, key) => {
  console.log(key, value);
});

// logs these on a new line:
// Orlando {"temp": 85, "condition": "Cloudy"}
// New York {"temp": 75, "condition": "Sunny"}
```



Comparing TypeScript Collections

Collection Name	Arrays	Tuples	Sets	Maps
Order type	Ordered and can reorder	Ordered by type position	Ordered by insertion order	Keys ordered by insertion order
How to declare	<code>type[]</code> or <code>Array<type></code>	<code>[type1, type2]</code>	<code>new Set<type>()</code>	<code>new Map<key type, value type>()</code>
Index style	Positional index, starting at 0	Positional index, starting at 0	No index, lookup by value	Index lookup by key
Duplicate values	Can contain duplicate values	Can contain duplicate values	Can not contain duplicate values	Keys unique, values can be duplicate
Length	Any length, can grow and shrink	Fixed length	Any length, can grow and shrink	Any length, can grow and shrink as keys are added or removed
Sortable	Sortable	Not sortable	Not sortable	Keys and values not sortable
Mutable	Mutable by default	Mutable by default	Mutable by default	Mutable by default
Immutable version	<code>readonly type[]</code> or <code>ReadonlyArray<type>()</code>	<code>readonly [type1, type2]</code>	<code>ReadonlySet<type>()</code>	<code>ReadonlyMap<key type, value type>()</code>